

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO1 – Precision (BL1, BL2, BL3)	Assessment:	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA	Total Marks:	100 (2 Questions × 50 Marks)
CO1 Statement:	<i>Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method. (BL1, BL2, BL3)</i>		
Student Name:	M Vishnu Vardhan	Reg. No.:	192472087
Section / Batch:	2024	Date:	22/07/2026

Instructions to Students

- Answer BOTH questions. Each question carries 50 Marks. Total: 100 Marks.
- Clearly show all steps in derivations and complexity analysis.
- Use proper asymptotic notation (Big-O, Big-Θ, Big-Ω) wherever required.
- Justify each step in recurrence solving using appropriate methods.
- Neat presentation and logical explanation are mandatory

Question Type	Focus Area	Questions	Marks
Application-Based Questions	Apply Master Theorem and asymptotic analysis to real-world scenarios	Q1 – Q2	Q1 –50 Q2 –50
GRAND TOTAL:		100 Marks	

SECTION A – APPLICATION BASED QUESTIONS

Q45. Application: Smart Grid Fault Detection System

A power grid monitoring system uses the recurrence $T(n) = 3T(n/2) + n^2$. Apply the Master Theorem to determine the time complexity. Clearly identify parameters and justify the selected case. Analyze performance for large-scale grid systems. Interpret efficiency using asymptotic notation.

Q46. Application: Speech Recognition Processing System

A speech processing system follows the recurrence $T(n) = 2T(n/2) + \sqrt{n}$. Apply the Master Theorem to determine the time complexity. Identify parameters and justify the case selection. Analyze how processing time increases with larger audio inputs. Interpret efficiency using asymptotic notation.

Code of Conduct

I M Vishnu Vardhan- 192472087 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student M Vishnu

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20%	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30%	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20%	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20%	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10%	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Marks Evaluation:

Question No.	Understanding of Problem Context (20%)	Application of Concepts (30%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (10%)	Total Marks
Q1 (50 Marks)						
Q2 (50 Marks)						
Overall Total (100)						

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q45. Recurrence Relation → Expansion → Pattern → Summation → Complexity

Concept Map (Extended)

Recurrence Relation



Expansion (unfold $T(n)$ repeatedly)



Pattern (observe recurring structure/term)



► Substitution Method (verify guessed pattern by induction)



Summation (sum all expanded terms, usually geometric/arithmetic series)



Complexity (closed-form Big-O expression)

Explanation: How Recurrence Expansion Reveals Patterns

A recurrence relation expresses the running time of an algorithm in terms of itself on smaller inputs, e.g.:

$$T(n) = 2T(n/2) + n$$

Step 1 – Expansion:

We repeatedly substitute the recurrence into itself:

- $T(n) = 2T(n/2) + n$
- $T(n) = 2[2T(n/4) + n/2] + n = 4T(n/4) + 2n$
- $T(n) = 4[2T(n/8) + n/4] + 2n = 8T(n/8) + 3n$
- After k expansions: $T(n) = 2^k T(n/2^k) + kn$

Step 2 – Pattern Recognition:

Looking at successive steps, we notice:

- The coefficient of T doubles each time $\rightarrow 2^k$
- The problem size shrinks by $2^k \rightarrow n/2^k$
- The added term grows linearly with $k \rightarrow kn$

This is the "pattern" — a general k -th term formula that predicts the entire expansion without manually writing every step.

Step 3 – Boundary Condition & Summation:

Expansion stops when $n/2^k = 1$, i.e., $k = \log_2 n$.

Substituting back:

$$T(n) = n \cdot T(1) + n \log_2 n = O(n \log n)$$

The summation step totals all the "extra work" (the n , $2 \times (n/2)$, $4 \times (n/4)$... terms) added at each level, often forming an arithmetic or geometric series that is summed using standard series formulas.

Extension: Substitution Method Link

The substitution method connects to "Pattern" as a verification step:

1. Guess the closed form suggested by the pattern (e.g., $T(n) = O(n \log n)$).
2. Assume it holds for smaller inputs: $T(n/2) \leq c(n/2)\log(n/2)$.
3. Prove by induction that it holds for n :

$$T(n) = 2T(n/2) + n \leq 2 \cdot c(n/2)\log(n/2) + n = cn \log(n/2) + n = cn \log n - cn + n \leq cn \log n \text{ (for } c \geq 1)$$
4. This confirms the guessed pattern rigorously, rather than relying only on informal expansion.

So the map extends as: Pattern $\rightarrow$ Substitution Method $\rightarrow$ Verified Closed Form

Justification: Importance of Pattern Recognition

- Avoids exhaustive expansion — instead of expanding to the base case every time, the k -th term formula gives an instant general expression.
- Enables generalization — once a pattern is found, it applies to an entire class of recurrences with similar structure (e.g., all $T(n) = aT(n/b) + f(n)$ forms $\rightarrow$ leads to Master Theorem).
- Reduces error — recognizing a series pattern lets us apply known summation formulas (arithmetic, geometric, harmonic) instead of ad hoc calculation.
- Bridges to formal proof — pattern recognition gives the *guess*, which substitution method or Master Theorem then *proves/confirms*.

Interpretation: How This Supports Solving Recurrences

Together, the pipeline forms a complete strategy:

Stage	Role
Expansion	Reveals the recurrence's internal structure
Pattern	Compresses expansion into a general k -th term
Substitution	Formally verifies the guessed pattern
Summation	Aggregates all cost terms into one expression
Complexity	Final Big-O/ Θ result for algorithm analysis

This shows recurrence-solving is not a single trick but a layered analytical method: informal exploration (expansion/pattern) followed by rigorous proof (substitution), ending in a usable complexity result that guides algorithm design decisions.

Q46. Divide and Conquer → Subproblems → Recurrence → Complexity

Concept Map (Extended)

Divide and Conquer



Subproblems (divide input into 'a' parts, each of size n/b)



Recurrence → $T(n) = a \cdot T(n/b) + f(n)$



— a = number of subproblems

— b = factor by which size shrinks



Complexity (solved via Master Theorem/recursion tree)

Mapping: How Problem Division Leads to Recurrence

Divide-and-Conquer algorithms work in three phases:

1. Divide — split the problem of size n into smaller subproblems.
2. Conquer — solve each subproblem recursively (this recursive call is exactly what generates the recurrence).
3. Combine — merge subproblem solutions, which costs additional time f(n).

Because the "conquer" step calls the same algorithm on smaller inputs, the total time naturally expresses itself as a recurrence:

$$T(n) = a \cdot T(n/b) + f(n)$$

Example — Merge Sort: divide array into 2 halves ($a = 2$, $b = 2$), sort each half recursively, merge in $O(n)$ time → $T(n) = 2T(n/2) + n$

So "division of the problem" directly becomes the recursive term $aT(n/b)$, and the "combine" step becomes $f(n)$.

Extension with Parameters a and b

Parameter	Meaning	Example (Merge Sort)	Example (Binary Search)
a	Number of subproblems created at each recursive call	a = 2	a = 1
b	Factor by which the problem size is divided	b = 2	b = 2
f(n)	Cost of divide + combine steps outside recursive calls	O(n) (merging)	O(1) (comparison)

General form: $T(n) = a \cdot T(n/b) + f(n)$, where $a \geq 1$, $b > 1$.

Justification: Role of a and b in Complexity

Using the Master Theorem, complexity depends entirely on the relationship between a, b, and f(n):

Compare f(n) with $n^{(\log_b a)}$:

- If $f(n) = O(n^{(\log_b a - \epsilon)}) \rightarrow T(n) = \Theta(n^{(\log_b a)})$ (recursive work dominates)

- If $f(n) = \Theta(n^{\log_b a}) \rightarrow T(n) = \Theta(n^{\log_b a} \cdot \log n)$ (balanced)
- If $f(n) = \Omega(n^{\log_b a + \epsilon}) \rightarrow T(n) = \Theta(f(n))$ (combine step dominates)

Why a and b matter:

- a controls how much recursive branching occurs — more subproblems mean more recursive work.
- b controls how fast the problem shrinks — larger b means fewer recursion levels (depth = $\log_b n$).
- The ratio between a 's growth and b 's shrinkage (captured in $n^{\log_b a}$) determines whether recursive cost or combine cost dominates the total complexity.

For merge sort: $a = 2, b = 2 \rightarrow n^{\log_2 2} = n^1 = n$, and $f(n) = n \rightarrow$ Case 2 $\rightarrow T(n) = \Theta(n \log n)$

Interpretation: How This Connects Design and Analysis

- Design phase (Divide and Conquer) determines *how* a problem is split — this is an algorithmic/creative decision (how many subproblems? how much smaller?).
- Analysis phase (Recurrence $\rightarrow$ Complexity) is a *mathematical consequence* of that design decision — once a and b are fixed by the design, the complexity is essentially determined.
- This shows a direct, causal link: algorithm design choices ($a, b, f(n)$) $\rightarrow$ recurrence relation $\rightarrow$ provable complexity bound.
- It also explains *why* some divide-and-conquer designs are better than others: e.g., choosing a smarter combine step (like in Strassen's matrix multiplication, which reduces a from 8 to 7) directly lowers overall complexity — proving that design and analysis are two sides of the same process, not separate activities.